🔍 7 Essential Data Quality Checks Every Data Team Should Know

✅ **Completeness**
Ensure no missing values in critical fields.

🔁 **Consistency**
Values follow the same format across tables/sources (e.g., date formats, country codes).

🧠 Accuracy
Data reflects the real-world truth (e.g., birth dates not in the future).

🔄 **Uniqueness**
No duplicates where uniqueness is expected (e.g., user IDs, email addresses).

🔗 Referential Integrity
Foreign keys correctly reference primary keys (e.g., orders link to valid customers).

⏰ **Timeliness**
Data is updated and reflects the current state (especially in streaming systems).

📏 **Validity**
Data conforms to business rules (e.g., age between 0–120, email format).

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, count, when, isnan, regexp_extract,
    to_date, current_date, datediff, length
)

spark = SparkSession.builder.master("local[*]").appName("DataQualityChecks").getOrCreate()

# Sample Data
data = [
    (1, "Alice", "alice@example.com", "2023-01-01", "US", 30),
    (2, None, "bob@example.com", "not-a-date", "UK", None),
    (3, "Charlie", "charlie@example", "2024-05-12", "US", 150),
    (4, "Alice", "alice@example.com", "2023-01-01", "US", 30),  # Duplicate
    (None, "Eve", "", "2023-04-01", "XX", -5)
]
columns = ["id", "name", "email", "signup_date", "country", "age"]

df = spark.createDataFrame(data, columns)

# 1. Completeness Check
print("=== Completeness ===")
df.select([
    count(when(col(c).isNull() | (col(c) == "") | isnan(c), c)).alias(f"{c}_nulls")
    for c in df.columns
]).show()

# 2. Consistency Check (email format)
print("=== Consistency (Email Format) ===")
df.withColumn("valid_email", regexp_extract("email", r'^[\w\.-]+@[\w\.-]+\.\w+$', 0)) \
  .select("email", "valid_email") \
  .show()

# 3. Accuracy Check (age range)
print("=== Accuracy (Age) ===")
df.filter((col("age") < 0) | (col("age") > 120) | col("age").isNull()).show()

# 4. Uniqueness Check (duplicate rows and IDs)
print("=== Uniqueness (Duplicate Rows) ===")
df.groupBy(df.columns).count().filter("count > 1").show()

print("=== Uniqueness (Duplicate IDs) ===")
df.groupBy("id").count().filter("count > 1").show()

# 5. Referential Integrity Check (valid countries)
print("=== Referential Integrity (Country) ===")
valid_countries = ["US", "UK", "IN"]
df.filter(~col("country").isin(valid_countries)).show()

# 6. Timeliness Check (signup_date freshness)
print("=== Timeliness (Signup Date) ===")
df = df.withColumn("signup_date_parsed", to_date("signup_date", "yyyy-MM-dd"))
df.withColumn("days_old", datediff(current_date(), "signup_date_parsed")).select("signup_date",
"days_old").show()

# 7. Validity Check (name length rule)
print("=== Validity (Name Length) ===")
df.filter(length(col("name")) < 2).show()
```

```python
from pyspark.sql import SparkSession
import great_expectations as ge
from great_expectations.dataset import SparkDFDataset
from pyspark.sql.functions import rand

spark = SparkSession.builder.master("local[*]").appName("GE_PySpark_Quality").getOrCreate()

# Sample skewed data
data = [
    (1, "Alice", "alice@example.com", "2023-01-01", "US", 30),
    (2, None, "bob@example.com", "not-a-date", "UK", None),
    (3, "Charlie", "charlie@example", "2024-05-12", "US", 150),
    (4, "Alice", "alice@example.com", "2023-01-01", "US", 30),
    (None, "Eve", "", "2023-04-01", "XX", -5)
]
columns = ["id", "name", "email", "signup_date", "country", "age"]
df = spark.createDataFrame(data, columns)

# Wrap with GE
df_ge = SparkDFDataset(df)

# 1. Completeness
df_ge.expect_column_values_to_not_be_null("id")
df_ge.expect_column_values_to_not_be_null("name")

# 2. Consistency (email format)
df_ge.expect_column_values_to_match_regex("email", r"^[\w\.-]+@[\w\.-]+\.\w+$")

# 3. Accuracy (age range)
df_ge.expect_column_values_to_be_between("age", 0, 120, allow_cross_type_comparisons=True)

# 4. Uniqueness (id)
df_ge.expect_column_values_to_be_unique("id")

# 5. Referential Integrity (country in list)
df_ge.expect_column_values_to_be_in_set("country", ["US", "UK", "IN"])

# 6. Timeliness (valid dates)
df_ge.expect_column_values_to_match_strftime_format("signup_date", "%Y-%m-%d")

# 7. Validity (name min length)
df_ge.expect_column_value_lengths_to_be_between("name", 2, 50)

# Run all checks
results = df_ge.validate()
print(results)
```

```
{
  "dq_checks": [
    {
      "column": "id",
      "check": "not_null",
      "enabled": true
    },
    {
      "column": "email",
      "check": "regex",
      "pattern": "^[\\w\\.-]+@[\\w\\.-]+\\.\\w+$",
      "enabled": true
    },
    {
      "column": "age",
      "check": "range",
      "min": 0,
      "max": 120,
      "enabled": true
    },
    {
      "column": "country",
      "check": "value_in_set",
      "values": ["US", "UK", "IN"],
      "enabled": true
    },
    {
      "column": "signup_date",
      "check": "date_format",
      "format": "yyyy-MM-dd",
      "enabled": true
    },
    {
      "column": "name",
      "check": "min_length",
      "min_length": 2,
      "enabled": true
    }
  ]
}
```

```python
import json
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, length, to_date, regexp_extract, lit

# Load config
with open("dq_config.json") as f:
    config = json.load(f)

# Start Spark
spark = SparkSession.builder.master("local[*]").appName("PlainPySparkDQ").getOrCreate()

# Sample Data
data = [
    (1, "Alice", "alice@example.com", "2023-01-01", "US", 30),
    (2, None, "bob@example.com", "not-a-date", "UK", None),
    (3, "Charlie", "charlie@example", "2024-05-12", "US", 150),
    (4, "Alice", "alice@example.com", "2023-01-01", "US", 30),
    (None, "Eve", "", "2023-04-01", "XX", -5)
]
columns = ["id", "name", "email", "signup_date", "country", "age"]
df = spark.createDataFrame(data, columns)

# Apply checks
for rule in config["dq_checks"]:
    if not rule.get("enabled", True):
        continue

    col_name = rule["column"]
    check = rule["check"]

    print(f"\n>>> Running check: {check} on column: {col_name}")

    if check == "not_null":
        df.filter(col(col_name).isNull()).show()

    elif check == "regex":
        pattern = rule["pattern"]
        df.filter(~(col(col_name).rlike(pattern))).show()

    elif check == "range":
        df.filter((col(col_name) < rule["min"]) | (col(col_name) > rule["max"])).show()

    elif check == "value_in_set":
        values = rule["values"]
        df.filter(~col(col_name).isin(values)).show()

    elif check == "date_format":
        date_format = rule["format"]
        df.filter(to_date(col(col_name), date_format).isNull()).show()

    elif check == "min_length":
        df.filter(length(col(col_name)) < rule["min_length"]).show()
```


--packages com.amazon.deequ:deequ:1.2.2-spark-3.3
from pyspark.sql import SparkSession
spark = SparkSession.builder \
.appName("DeequDQ") \
.config("spark.jars.packages", "com.amazon.deequ:deequ:1.2.2-spark-3.3")
\ .getOrCreate()

```python
from pyspark.sql import SparkSession
from py4j.java_gateway import java_import

# Start Spark with Deequ
spark = SparkSession.builder \
    .appName("DeequDQ") \
    .config("spark.jars.packages", "com.amazon.deequ:deequ:1.2.2-spark-3.3") \
    .getOrCreate()

# Import Deequ Java classes
java_import(spark._jvm, "com.amazon.deequ.*")
java_import(spark._jvm, "com.amazon.deequ.analyzers.*")
java_import(spark._jvm, "com.amazon.deequ.checks.*")
java_import(spark._jvm, "com.amazon.deequ.constraints.*")
java_import(spark._jvm, "com.amazon.deequ.VerificationSuite")
java_import(spark._jvm, "com.amazon.deequ.VerificationResult")

# Sample Data
data = [
    (1, "Alice", "alice@example.com", "2023-01-01", "US", 30),
    (2, None, "bob@example.com", "not-a-date", "UK", None),
    (3, "Charlie", "charlie@example", "2024-05-12", "US", 150),
    (4, "Alice", "alice@example.com", "2023-01-01", "US", 30),
    (None, "Eve", "", "2023-04-01", "XX", -5)
]
columns = ["id", "name", "email", "signup_date", "country", "age"]
df = spark.createDataFrame(data, columns)

# Create a Deequ Check
check = (
    spark._jvm.com.amazon.deequ.checks.Check(spark._jvm.com.amazon.deequ.checks.CheckLevel.Error(),
"Data Quality Check")
        .isComplete("id")  # Completeness
        .isUnique("id")    # Uniqueness
        .hasMinLength("name", lambda x: x >= 2)  # Validity
        .hasPattern("email", spark._jvm.com.amazon.deequ.patterns.Patterns.EMAIL())  # Consistency
        .isContainedIn("country", spark._jvm.scala.collection.JavaConverters.asScalaBuffer(["US", "UK",
"IN"]).toSet())  # Referential Integrity
        .hasDataType("signup_date",
spark._jvm.com.amazon.deequ.analyzers.types.ConstrainableDataTypes.String)  # Timeliness placeholder
        .isNonNegative("age")  # Accuracy (basic)
        .hasMax("age", lambda x: x <= 120)  # Accuracy (range upper bound)
)

# Run Deequ Verification
result = (
    spark._jvm.VerificationSuite()
        .onData(df._jdf)
        .addCheck(check)
        .run()
)

# Pretty print results
result_df =
spark._sc._jvm.com.amazon.deequ.VerificationResult.checkResultsAsDataFrame(spark._jsparkSession,
result)
result_df.show(truncate=False)
```